Y.A.C.I.N.E. Yet Another Cognitive Intelligent Neural Engineer

Mission

Build a fully local, privacy-first autonomous software engineering agent capable of understanding software projects, planning implementations, writing production-ready code, executing it locally, debugging itself, modifying existing codebases, managing projects, using tools, searching the internet, and continuously improving until every requested task is completed successfully.

The system should behave like a senior software engineer—not an autocomplete assistant.

Everything must execute locally on Windows 11 with GPU acceleration.

The agent must never stop after generating code.

Instead, it must continue working until the requested task has actually been completed and verified.

Overall Design Philosophy

This is NOT a chatbot.

This is NOT an IDE extension.

This is NOT an LLM wrapper.

This is an autonomous software engineer.

It should think.

Plan.

Execute.

Observe.

Repair.

Retry.

Validate.

Then deliver.

The entire architecture must revolve around autonomous execution.

Primary Objectives

The system should be capable of:

understanding entire repositories

modifying existing projects

generating complete applications

fixing bugs

writing tests

executing tests

installing dependencies

creating documentation

running commands

debugging runtime failures

browsing the internet

reading documentation

searching GitHub

creating files

editing files

deleting files

moving files

running Docker

executing Python

executing Node

executing Java

executing C++

compiling projects

building applications

generating APIs

managing databases

using MCP servers

calling local tools

interacting with Git

interacting with terminals

understanding project architecture

remembering previous conversations

Hardware Target

Primary Target:

Windows 11

RTX 2080 Super

32GB RAM

Ryzen CPU

Runs completely offline except optional internet search.

AI Model Support

Must support interchangeable models.

Primary:

Qwen2.5-Coder

Future Support:

DeepSeek

Qwen3-Coder

Llama

Codestral

GPT OSS models

Phi

Gemma

Mistral

Model providers:

Ollama

LM Studio

llama.cpp

vLLM

OpenAI Compatible APIs

User can switch models without changing code.

Architecture

Use a modular architecture.

Example:

Core/

Agents/

Planner/

Reasoner/

Memory/

Execution/

Filesystem/

Internet/

IDE/

Terminal/

Tools/

Testing/

UI/

API/

Config/

Telemetry/

Logs/

Models/

Database/

Security/

Extensions/

Plugins/

MCP/

Providers/

Utilities/

Multi-Agent System

Instead of one LLM, build specialized agents.

Examples:

Project Manager Agent

Responsible for:

task decomposition

milestone creation

prioritization

Architect Agent

Responsible for:

software architecture

design patterns

scalability

Research Agent

Responsible for:

documentation search

StackOverflow

GitHub

official docs

Coder Agent

Responsible for:

implementation

Reviewer Agent

Responsible for:

code review

optimization

readability

Debugger Agent

Responsible for:

analyzing stack traces

fixing crashes

Test Engineer

Responsible for:

unit tests

integration tests

regression tests

Security Agent

Responsible for:

vulnerabilities

secrets

dependency analysis

DevOps Agent

Responsible for:

Docker

CI/CD

deployment

Documentation Agent

Responsible for:

README

API docs

comments

changelogs

These agents collaborate automatically.

Autonomous Development Loop

Every task follows this lifecycle:

Understand Request

↓

Analyze Project

↓

Build Context

↓

Research Missing Information

↓

Generate Plan

↓

Wait for User Approval (optional)

↓

Implement

↓

Generate Tests

↓

Execute

↓

Observe

↓

Read Errors

↓

Self Debug

↓

Modify Code

↓

Re-run

↓

Repeat Until Green

↓

Review

↓

Optimize

↓

Generate Documentation

↓

Commit Changes

↓

Deliver

The loop never stops after the first code generation.

Planning Engine

Before writing code the AI must generate:

Requirements

Architecture

Dependencies

Risks

Edge Cases

Performance Considerations

Security Considerations

Estimated Files

Implementation Steps

Testing Strategy

Rollback Strategy

Context Engine

The system should understand:

Entire repositories

Folder structures

Imports

Dependencies

Git history

Existing architecture

Coding style

Naming conventions

README

Documentation

Configuration files

Environment variables

Database schema

API routes

Package manager

Framework

Languages

Everything becomes searchable.

Memory System

Implement persistent memory.

Remember:

Previous projects

Coding preferences

Frequently used frameworks

Favorite libraries

User style

Completed tasks

Failures

Successful fixes

Store memories locally.

Support vector search.

Use embeddings.

Internet Research

The Research Agent should:

Search Google

Search GitHub

Search official documentation

Search MDN

Search Microsoft Docs

Search Python Docs

Search npm

Search PyPI

Search Reddit

Search StackOverflow

Search HuggingFace

Search Docker Docs

Search Rust Docs

Search Java Docs

Search C++ references

Automatically summarize findings.

Rank sources by reliability.

Never hallucinate APIs when documentation is available.

Tool System

The AI must call tools instead of pretending.

Tools include:

Terminal

PowerShell

CMD

Python

Git

Docker

Browser

Filesystem

HTTP Requests

Database

SQLite

PostgreSQL

MySQL

Redis

VS Code

Chrome

Clipboard

Calculator

Image tools

PDF parser

OCR

Speech-to-text

Text-to-speech

Plugin system

MCP servers

Everything should be tool-driven.

Filesystem

Capabilities:

Create files

Delete files

Rename files

Move files

Read files

Bulk edit

Search code

Regex replacement

Directory analysis

Safe overwrite

Version backup

Undo operations

Execution Sandbox

Never execute directly inside project.

Create isolated workspaces.

Use:

Temporary directories

Virtual environments

Docker (optional)

Subprocess

Timeout protection

Memory limits

CPU limits

Capture:

stdout

stderr

return codes

execution time

resource usage

Self-Healing Engine

When execution fails:

Capture exception

Capture traceback

Identify root cause

Search related documentation if needed

Rewrite affected functions only

Preserve working code

Retry automatically

Repeat until success or retry limit

Generate explanation of fixes

Testing System

Automatically generate:

Unit tests

Integration tests

Smoke tests

Regression tests

Edge case tests

Stress tests (optional)

Coverage report

Execute automatically.

Repair failing tests.

Code Quality

Automatically enforce:

Black

Ruff

ESLint

Prettier

Type checking

Security scanning

Linting

Dead code detection

Unused imports

Formatting

Git Integration

Capabilities:

Initialize repositories

Branch management

Commit

Push

Pull

Merge

Conflict resolution assistance

Commit message generation

Rollback

Diff visualization

Database Support

SQLite

MySQL

PostgreSQL

MongoDB

Supabase

Firebase

Automatic schema discovery.

Migration generation.

Query optimization.

Supported Languages

Python

JavaScript

TypeScript

Java

C#

Go

Rust

C

C++

PHP

Kotlin

Swift

Dart

HTML

CSS

SQL

Bash

PowerShell

Supported Frameworks

React

Next.js

Vue

Angular

Flutter

FastAPI

Flask

Django

Express

Spring Boot

ASP.NET

Laravel

Node.js

Electron

Tauri

PySide

Qt

Desktop Interface

Modern UI similar to:

Claude Desktop

Cursor

Codex

VSCode

ChatGPT Desktop

Features:

Dark Mode

Light Mode

Resizable Panels

Tabbed Chats

Project Explorer

Terminal

Logs

Execution Monitor

Memory Viewer

Task Queue

Agent Status

Settings

Model Selector

Token Usage

GPU Usage

CPU Usage

RAM Usage

Download Progress

File Preview

Markdown Renderer

Syntax Highlighting

Chat Features

Artifacts

Code blocks

Mermaid diagrams

Markdown preview

Image support

PDF upload

Folder upload

Drag & drop

Conversation search

Pinned chats

Bookmarks

Conversation export

Security

Everything local.

Never upload files without permission.

Permission system:

Read file

Write file

Execute command

Delete file

Internet access

Git push

Each permission configurable.

Performance

Streaming responses

Parallel agents

Caching

Incremental indexing

Lazy loading

Background indexing

GPU acceleration

Asynchronous architecture

Plugin Architecture

Create a plugin SDK.

Plugins can add:

Tools

Models

Providers

UI panels

Commands

Agents

Workflows

Themes

API

Expose REST and WebSocket APIs.

Allow external applications to interact with Y.A.C.I.N.E.

Support authentication and API keys.

Logging

Every action should be logged.

Reasoning summary

Tool calls

Execution logs

Errors

Fixes

Retries

Performance metrics

Token usage

Execution history

Development Phases

Phase 1

Core architecture

Model integration

Local chat

Tool framework

File management

Phase 2

Autonomous planning

Multi-agent orchestration

Execution sandbox

Testing engine

Phase 3

Self-healing loop

Repository indexing

Persistent memory

Git integration

Phase 4

Internet research

Documentation retrieval

MCP support

Plugin SDK

Phase 5

Desktop UI

Advanced workflows

Performance optimization

Security hardening

Phase 6

Voice interaction

Vision capabilities

Long-term memory improvements

Collaborative multi-agent execution

Expected Result

The final product should feel like having a senior software engineer permanently available on the local machine—one that understands entire codebases, researches when necessary, plans before coding, executes every change, validates the results through automated testing, repairs failures autonomously, and delivers only verified, production-ready implementations. It should be modular, extensible, privacy-first, performant on consumer hardware such as an RTX 2080 Super, and designed to evolve into a complete autonomous AI development platform rather than a simple coding assistant.